

Steps in Eclipse IDE for creating a Maven based dynamic web app with Spring MVC ,Hibernate n MySql (without Spring boot)

0. Copy n Import "Template Projects\spring_mvc_template" , in your workspace , as existing maven project

1.

All XML templates are present in : day11-data\day11_help\spring-hibernate-templates folder.
Keep the folder open for your quick reference.

2. Copy welcome-file-list , servlet & servlet-mapping tags from the web.xml under templates folder

Meaning : To Configure spring supplied Front controller(o.s.w.s.DispatcherServlet) to intercept any request from any client, in web.xml.

Detailed explanation of tags

`<url-pattern>/</url-pattern>` => any request received from any client

`<servlet-name>spring</servlet-name>` => can be replaced by any name

`<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>` => spring supplied Front Controller

`<load-on-startup>1</load-on-startup>` => Web container (WC) will start the life cycle of DispatcherServlet , at the web app deployment time.

DispatcherServlet will create instance of XmlWebApplicationContext

Job of D.S (DispatcherServlet)

To start the SC (spring container) in web app.

(Currently it will create instance of XmlWebApplicationContext)

3. Copy spring-servlet.xml from templates folder , under <WEB-INF> of your web app.
Initially comment or remove this line.

`<import resource="classpath:/hibernate-persistence.xml"/>`

Meaning

spring-servlet.xml => master configuration xml file for starting SC.

D.S(DispatcherServlet) reads this config file , @ web app deployment time , to start SC , within a web app. (represented by i/f WebApplicationContext --sub i/f of ApplicationContext)

Default location of this master configuration xml file--<WEB-INF>

Default name -- `servletName-servlet.xml`

Since we have supplied , servlet-name , in web.xml as spring , in our case it's spring-servlet.xml

Detailed explanation of tags

3.1 `<context:annotation-config />` => Enables class internal annotations.

3.2 `<context:component-scan base-package="com.app"/>` => SC will scan only com.app & its sub packages for spring beans.

3.3 `<mvc:annotation-driven/>` => Enables annotation based MVC support (i.e enables automatic population of HandlerMapping bean using @RequestMapping annotation in Controller beans)

Regarding HandlerMapping bean

o.s.w.s.HandlerMapping --i/f

Implementation class -- RequestMappingHandlerMapping

Consists of a map , populated by SC @ web app deployment time.

Key --value of @RequestMapping annotation, in controller bean

Value --F.Q controller cls name + method name.

```
3.4 <bean id="viewResolver"  
class="org.springframework.web.servlet.view.InternalResourceViewResolver"  
p:prefix="/WEB-INF/views" p:suffix=".jsp" p:viewClass="org.springframework.web.servlet.view.JstlView"  
>
```

Meaning : Configure , spring supplied ViewResolver bean , for translation from logical view name to actual view name (by wrapping it in prefix n suffix)

More Details about ViewResolver Bean

o.s.w.s.ViewResolver --i/f

Implementation class : o.s.w.s.view.InternalResourceViewResolver

It uses setter based D.I .

ViewResolver => InternalResourceViewResolver
HandlerMapping => RequestMappingHandlerMapping

It has 3 properties

prefix = /WEB-INF/views

suffix = .jsp

viewClass = JstlView

eg : If logical view name is "/welcome"

Actual view name will be : /WEB-INF/views/welcome.jsp

This completes configuration steps .

4. Create request handling controller, to test MVC flow.(under com.app.controller pkg)

eg : HelloController

Mandatory Annotations used -- @Controller --class level.

@RequestMapping --method level annotation

(To map client requests onto specific controller class's specific method)

eg : @Controller

```
public class HelloController {
```

```
@RequestMapping("/hello1")
```

```
public String sayHello1()
```

```
{  
    return "/welcome";  
}
```

```
}
```

```
}
```

Explanation : HelloController bean is singleton n eager (i.e it's single instance will be created at web app deployment time)

So for this controller , what will be the entry in HandlerMapping bean, created at web app deployment time ?

Key -- /hello1

Value -- F.Q controller class name + method name

5. To Test Spring MVC flow --

5.1 Add index.jsp in WebContent , with a link.

```
<a href="hello1">Test Spring MVC Flow</a>
```

Note : href of the anchor tag MUST match will the value of the @RequestMapping annotation.

5.2 Add a welcome.jsp under a folder <WEB-INF>/views/ & add a welcome message.

6. Run web application

R click --run on server

Troubleshooting tips : check on server console for the following :

6.1 Initializing Spring DispatcherServlet 'spring'

6.2 org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'spring'

6.3 in constr of com.app.controller.HelloController

in init

This indicates that spring based web app is up n running.

Go to client browser & test it!